Scrivere codice: Visual Studio Code

Dario Pescini

Università degli Studi di Milano-Bicocca

Dipartimento di Statistica e Metodi Quantitativi

dario.pescini@unimib.it

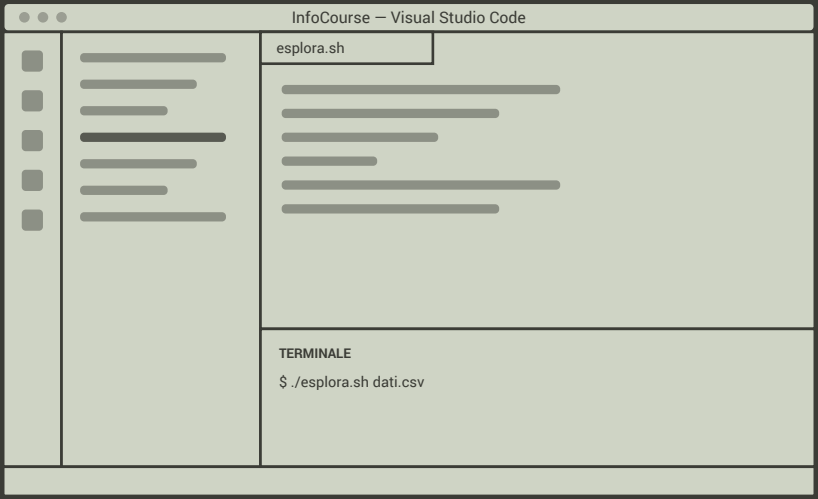
Dalla riga di comando all'editor

Finora i file li abbiamo creati con `echo` e guardati con `cat`. Va bene per una riga.

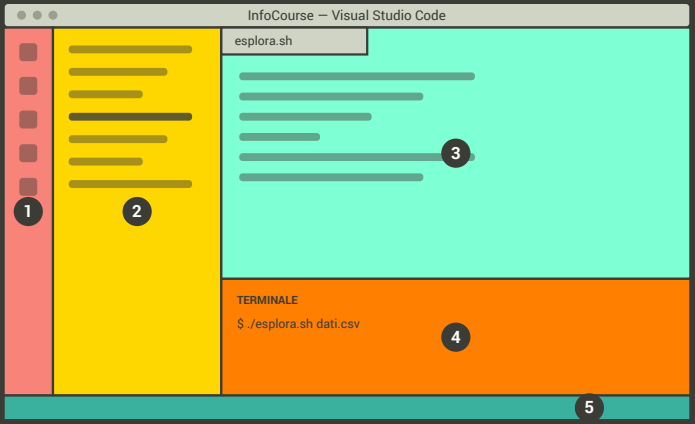
- uno script di dieci righe si scrive, si rilegge e si **corregge**: con `echo` ogni correzione è riscrivere tutto
- l'editor **colora** il codice, e il colore è la prima diagnosi: se una stringa resta aperta, il resto della riga cambia colore
- l'editor mostra quello che la shell **non fa vedere** — spazi in fondo alle righe, tabulazioni, fine riga di Windows

Sulla macchina del corso l'editor è Visual Studio Code. Le stesse idee valgono per qualunque altro: cambia dove stanno i bottoni.

La finestra, appena aperta



Cinque regioni, e non cambiano mai



1 barra delle attività **2** Explorer **3** Editor
4 Pannello (terminale) **5** barra di stato

1. La barra delle attività

La colonna di icone a sinistra: cambia **cosa** mostra l'Explorer accanto.

- **Explorer** — i file della cartella aperta. È quella che userai sempre
- **Cerca** — cerca in *tutti* i file della cartella, non solo in quello aperto: è `grep` con l'interfaccia
- **Controllo del codice sorgente** — git, quando lo vedremo
- **Estensioni** — i componenti aggiuntivi. Quello di Python servirà da `L03` in poi

Si apre e si chiude con `Ctrl+B`: serve quando lo schermo è piccolo.

2. Explorer: si apre una cartella, non un file

La prima cosa che si fa, ogni volta

`File` ▸ `Apri cartella…` e si sceglie la cartella in cui si lavora. **Non** il singolo file.

Perché cambia tutto:

- l'Explorer diventa l'**albero** di quella cartella: la stessa cosa che `ls` mostra un livello alla volta
- il terminale integrato **parte lì dentro**, quindi i percorsi relativi funzionano senza `cd`
- salvando un file nuovo, propone **quella** cartella

Chi arriva da altri ambienti cerca il «progetto»: qui non c'è, e non serve. La cartella aperta è il progetto.

3. L’editor: schede, colore, e un pallino

- ogni file aperto è una **scheda** in alto
- il **colore** del testo viene dal linguaggio, che VS Code deduce dall’**estensione**: senza `.sh` o `.py` scrivi in bianco e nero
- premendo `Tab` durante la scrittura, propone il completamento

Il pallino al posto della X

Un file modificato e **non salvato** ha un pallino sulla sua scheda. Se lanci uno script e vedi il comportamento di prima, guarda lì: stai eseguendo la versione **su disco**, non quella che hai davanti. `Ctrl+S`.

4. Il pannello: il terminale è lo stesso di prima

`Terminale` ▸ `Nuovo terminale`, oppure `Ctrl+ù` sulle tastiere italiane.

- è la **stessa shell** della lezione: stessi comandi, stesso `pwd`, stesso prompt
- parte nella **cartella aperta**, che è il motivo per cui si apre una cartella e non un file
- accanto al terminale ci sono `PROBLEMI` e `OUTPUT`: il primo elenca gli errori che l'editor vede *prima* di eseguire

```
$ pwd
/home/studente/InfoCourse/01_AmbienteDiLavoro
$ ./esplora.sh data/2009-2013_iscritti.csv ';'
```

5. La barra di stato: la riga che nessuno guarda

In basso a destra, quattro informazioni che contano **tutte**:

- `Ln 12, Col 34` — dove sta il cursore. Gli errori dicono la riga: questa è la riga
- `Spaces: 4` — indentazione a spazi o a tabulazioni. In Python l'indentazione **è** il programma
- `UTF-8` — la codifica. È quella della lezione sui byte
- `LF` o `CRLF` — il fine riga. `CRLF` vuol dire che il file viene da Windows, e cliccandoci sopra si converte

Si clicca su ciascuna per cambiarla. È il posto dove si risolvono in un secondo errori che altrimenti costano mezz'ora.

Creare un file, e salvarlo dove vuoi tu

1. `File` ▸ `Nuovo file di testo`, **o** `Ctrl+N`
2. `Ctrl+S` **subito**: si apre la finestra del nome
3. il nome **con l'estensione** — `esplora.sh`, non `esplora`
4. controlla la **cartella** proposta in alto: è lì che finirà

Dal terminale integrato, la verifica che non costa nulla:

```
$ pwd
$ ls -l esplora.sh
-rw-r--r-- 1 studente studente 812 22 set 14:10 esplora.sh
```

Le scorciatoie che servono davvero

Salva	`Ctrl+S`
Annulla / rifai	`Ctrl+Z` / `Ctrl+Shift+Z`
Commenta o decommenta le righe scelte	`Ctrl+/`
Duplica la riga	`Shift+Alt+Giù`
Sposta la riga su o giù	`Alt+Su` / `Alt+Giù`
Cancella la riga	`Ctrl+Shift+K`
Vai a un file per nome	`Ctrl+P`
Cerca nel file / in tutti i file	`Ctrl+F` / `Ctrl+Shift+F`
Apri o chiudi il terminale	`Ctrl+ù` (tasto del backtick)
Palette dei comandi	`Ctrl+Shift+P`

L'ultima vale per tutte le altre: ci scrivi una parola di quello che vuoi fare e lui propone il comando **con la sua scorciatoia accanto**.

Quello che non si vede, e fa sbagliare

Tre errori visti in questa lezione, tutti fatti di caratteri che non si vedono:

- il codice `T1` negli iscritti è largo tre caratteri: `T1` più uno **spazio**. Chi confronta con `T1` non trova nulla
- l'ultima colonna del meteo finisce con un tag HTML, e il valore non è un numero ma **testo**
- un file salvato su Windows ha **due** caratteri di fine riga, e il secondo entra dentro l'ultimo dato

Dalla shell si vedono con `cat -A`. Nell'editor si possono vedere sempre, e conviene.

Accenderli: dalle impostazioni

`File` ▸ `Preferenze` ▸ `Impostazioni`, o `Ctrl+virgola`.

Nella casella di ricerca si scrive il nome dell'impostazione e si cambia il valore:

`render whitespace`	da `selection` a `all`
`render control characters`	spuntato

Quello che NON va spuntato

`trim trailing whitespace` toglie gli spazi in fondo alle righe **quando salvi**. Su un file di dati è un disastro: negli iscritti il codice `T1` ha uno spazio che fa parte del dato, e salvando il file lo perderesti senza accorgertene. Mostrare sì, **modificare no**.

Accenderli: dal file delle impostazioni

Lo stesso risultato, ma scritto: `Ctrl+Shift+P`, poi `Preferences: Open User Settings (JSON)`.

```
{
    "editor.renderWhitespace": "all",
    "editor.renderControlCharacters": true,
    "files.eol": "\n",
    "editor.rulers": [80]
}
```

Le ultime due: i file **nuovi** nascono con il fine riga di Unix, e una riga verticale a colonna 80 ricorda dove il codice diventa illeggibile. Nessuna di queste impostazioni cambia i file che apri: le due prime li disegnano diversamente, non li riscrivono.

Cosa si vede, dopo

- ogni **spazio** diventa un puntino grigio a mezza altezza
- ogni **tabulazione** diventa una freccia che arriva alla colonna successiva: a colpo d'occhio si distingue da quattro spazi
- i **caratteri di controllo** — quelli che `cat -A` mostra come `^M` — compaiono come un rettangolino con dentro una sigla

Perché tenerli accesi sempre

Dopo due giorni non li noti più, e nel frattempo hai smesso di perdere tempo su un errore che «non c'è». È la stessa ragione per cui la barra di stato dice `UTF-8` e `LF`: l'invisibile va reso visibile **prima** che dia problemi, non dopo.

I dati si guardano con l'editor, non con Excel

Un `.csv` è un file di testo. Excel e Word non te lo mostrano: ti mostrano la **loro interpretazione**, e se salvi la scrivono sul file.

- gli **zeri iniziali** spariscono: il codice ateneo `00101` diventa il numero `101`
- le **date** vengono reinterpretate secondo la lingua del sistema, e `2011-1-1` può tornare in un altro formato
- il **separatore** cambia da punto e virgola a virgola, o viceversa, secondo le impostazioni locali

🔑 La regola

Per **guardare** i dati: l'editor, o la shell con `head` e `cut`. Per **modificarli**: un programma che scrivi tu, che legge un file e ne scrive un altro. L'originale non si tocca mai.

Eseguire quello che hai scritto

Dal terminale integrato, che è già nella cartella giusta:

```
$ bash esplora.sh data/2009-2013_iscritti.csv ';'   # sempre
$ chmod +x esplora.sh                               # una volta sola
$ ./esplora.sh data/2009-2013_iscritti.csv ';'      # da qui in poi
```

- `bash script` avvia una shell e le passa il file: funziona **sempre**, anche senza permessi
- `./script` chiede al sistema di eseguirlo: servono il permesso `x` e la prima riga `#!/usr/bin/env bash`
- il `./` non è un capriccio: senza, la shell cerca il nome fra i programmi installati, e la cartella corrente **non** è fra quelli

Il punto della lezione

- si apre **la cartella**, non il file: da lì discendono l'albero, il terminale e i percorsi relativi
- il terminale integrato è **la stessa shell**, a un centimetro dal testo che stai scrivendo
- la barra di stato dice **codifica e fine riga**: due errori su tre si diagnosticano lì
- i caratteri invisibili si accendono **una volta** e restano accesi

Il resto — completamento, estensioni, debugger — si impara quando serve, e la palette dei comandi è il modo di trovarlo senza cercarlo.